

Soteria

Trust, but diff



Machine Learning Project Proposal

Tom Goncin
Mathias Guterres Bergmann
University of Regina
September 23rd, 2026

Table of Contents

Abstract	3
Problem Definition	3
Research Questions	4
Objectives	4
Dataset & Data Preparation	5
Proposed Machine Learning Approaches	6
Evaluation Strategy	6
Evaluation Strategy	7
Project Boundaries	7
Expected Outcomes	8
Group Responsibilities	8
References	8

Abstract

Writing software requires constant updates— the addition of features, fixing bugs, and refactoring code. While the intention of these updates is to maintain the systems ability to stay sound and complete, they are notorious for accidentally introducing new security flaws, modifying code tied to what currently exists or even in some cases creating a cascading effect of errors.

Identifying these security-relevant changes is a major challenge. Traditional security checks, such as manually reviewing and static analysis rely on rigid rules and human expertise. While helpful, it doesn't always catch every risk. This project explores through machine learning an algorithm to identify "security-fix commits"—code changes that resolve known vulnerabilities. This objective will be achieved through analyzing the public CVEfixes dataset and comparing vulnerability fixes against the standard, non-security commits from the same repositories.

The goal is to determine if machine learning algorithms have the capability to identify specific patterns in the code to tell the difference between routine updates and critical security patches.

Problem Definition

The core issue tackled is how difficult it is to spot security patches when developers don't explicitly label them. Repositories process thousands of commits, making manual inspection impossible. Machine learning may be the answer that aids users in the automation of this process through recognizing historical patterns.

However, software drastically changes across different projects and languages. Furthermore, security datasets are notoriously unbalanced, often lacking context, and biased toward highly publicized bugs. This research project shall test how well various machine learning models handle this messy, real-world data.

Scope limitation:

The project's scope is limited to strictly looking at commits that fix documented vulnerabilities. Due to time constraints the project will not predict when a vulnerability is initially introduced, or have built a real-time monitoring tool, as well neither guaranteeing bug-free code. Detecting the commits that *cause* vulnerabilities is left for future work.

Research Questions

Can machine learning accurately distinguish security-fix commits from regular commits using change-level and source-code features?

Secondary Research Questions:

- Which code features give the strongest signal for distinguishing security fixes?
- How do different machine learning algorithms compare in this environment?
- How does feature selection impact the results?
- What are the practical limits of using historical datasets with artificially sampled negative examples?

Objectives

The team members will pursue the following goals:

1. Review current literature on machine learning in relation to software security.
2. Clean and preprocess the CVEfixes dataset.
3. Extract relevant code features, ensuring data used would be of that available *before* a fix is officially disclosed.
4. Build a balanced dataset by sampling regular commits from the same repositories as the security fixes.
5. Train and test several machine learning classification models.
6. Compare algorithm performance using standardized metrics.
7. Analyze where the models fail and identify any data biases.
8. Draw conclusions based on quantitative results, rather than inherently assuming machine learning algorithms will outperform traditional tools.

Dataset & Data Preparation

Data used in this study will be attained from CVEfixes, a public dataset linking software vulnerabilities to their exact fix commits. The dataset and its tools are hosted at <https://github.com/secureIT-project/CVEfixes>.

Data analyzed:

- Affected files, functions, and methods.
- Lines of code added and removed.
- Programming languages and source-code characteristics.
- Repository and commit metadata.
- Vulnerability identifiers (strictly for post-analysis, not as training features).

To keep the project manageable, the team will inhibit a limit in the quantity of repositories used between fifty and one hundred, and in one language, Python. Duplicated commits such as forks and backports will be filtered out of the assessment.

Important Data Consideration & Class Segregation

Since CVEfixes only tracks positive examples (the actual fixes), the projects team must create its own negative examples. These will segregate the dataset into two distinct classes:

Positive Class	Commits explicitly linked to fixing a documented CVE.
Negative Class	Ordinary commits that do not fix any known vulnerabilities, sampled directly from the exact same repositories as the positive examples.

Sampling from identical repositories will ensure the model learns to identify security features rather than just memorizing repository habits. This study acknowledges that some of the labeled "negative" commits may be hidden, undocumented security fixes; which is a standard limitation when dealing with repository data.

Proposed Machine Learning Approaches

The algorithms listed below will be those tested in order to identify what works best:

Algorithm	Purpose
Logistic Regression	A simple, easily interpretable baseline model.
Random Forest	Effective for handling nonlinear data and ranking feature importance.
Support Vector Machine	Strong for classifying highly complex, multidimensional data.
Gradient Boosting	A powerful ensemble method for comparison.

Initially beginning with a basic baseline, predicting solely based on the size of the change to analyze if the more complex machine learning models add measurable value.

Evaluation Strategy

While the final prediction will happen at the **commit level**, features will be pulled from the most specific scope, functions and methods, and sequentially work towards files and in turn entire commits.

Key features included:

- Number of modified functions or methods
- Number of files modified
- Lines added, deleted, and overall change size.
- Token-level code changes
e.g., TF-IDF over the code diff
- Commit message keywords
Tested separately to identify if text explains more than the code itself.

Crucially, any meta-information that wouldn't exist at prediction time will be stripped out. Information such as CVE numbers or CWE tags that prevent the model from "cheating" (data leakage).

Evaluation Strategy

Due to security data being heavily imbalanced, standard accuracy isn't enough. Success must be measured using Precision, Recall, F1-score, and PR-AUC (Precision-Recall Area Under Curve). Additionally, handling class imbalances by weighing or resampling the training data.

Strict Time-Based Split

Using Pareto's Principle, an 80:20 time-based split on the **commit** date will be enforced instead of randomly shuffling the data. This mirrors the behavior of a real CI/CD pipeline—training on past code to evaluate future changes.

Reporting will be conducted through the results compared side-by-side with a standard random split. Comparing the two will highlight any instances where the model is simply memorizing data (inflating the random split), rather than genuinely learning to generalize to new code. Wherever possible, the data will be split by repository to see how well the models perform on entirely unfamiliar projects.

Project Boundaries

Included in the project	Excluded from the project
Using public vulnerability data	Manually collecting or mining new vulnerabilities
Data preprocessing & explicit positive/negative class segregation	Building a commercial tool or real-time monitoring system
Feature engineering scaling from the function to the commit level	Deploying a live CI/CD plugin
Implementation & comparison in machine learning algorithms	Guaranteeing the discovery of unknown bugs
Side-by-side evaluation of time-based versus random data splits	Detecting the commits that <i>introduce</i> vulnerabilities.

Expected Outcomes

Towards the end of the term, this project is expected to deliver a fully preprocessed dataset, a detailed feature analysis, working machine learning model(s), and a final report comparing performances. Furthermore, whether the models succeed brilliantly or struggle due to data limits, the project will provide valuable insights into the viability of ML in this space.

Group Responsibilities

Both parties working on the project share equal responsibilities, and are not limited to these roles listed below:

- Project design and methodology
- Interpretation of results
- Final report and presentation
- Machine learning model implementation
- Model training and hyperparameter experimentation
- Evaluation using Precision, Recall, F1-score, and PR-AUC
- Analysis and visualization of experimental results
- Literature review and state-of-the-art analysis
- CVEfixes dataset preprocessing and data cleaning
- Feature extraction and engineering
- Documentation of dataset preparation

References

The methodology used is backed by recent research in vulnerability mining. For example, Wang et al. (2024) highlights the importance of untangling code patches to build clean and reliable datasets. Similarly, Akhoundali et al. (2024) demonstrated that expanding repository data significantly boosts the baseline performance of ML detection models.

- Akhoundali, J., Nouri, S. R., Rietveld, K., & Gadyatskaya, O. (2024). MoreFixes: A Large-Scale Dataset of CVE Fix Commits Mined through Enhanced Repository Discovery. *Proceedings of the 20th International Conference on Predictive Models and Data Analytics in Software Engineering*, 42–51.
- Wang, X., Hu, R., Gao, C., Wen, X.-C., Chen, Y., & Liao, Q. (2024). ReposVul: A Repository-Level High-Quality Vulnerability Dataset. *arXiv*.

- Bhandari, G., Naseer, A., & Moonen, L. (2021). CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '21)*.
<https://doi.org/10.1145/3475960.3475985>
- Sabetta, A., & Bezzi, M. (2018). A Practical Approach to the Automatic Classification of Security-Relevant Commits. *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*.
- Sun, X., et al. (2020). HERMES: Using Commit-Issue Linking to Detect Vulnerability-Fixing Commits. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.